hexens x UFARM DIGITAL

Security Review Report for UFarm.Digital

September 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Lack of frozen balance consideration in the withdrawal settlement flow
    - No cap on the total raise
    - forcedTransfer desyncs frozenTokens
    - Redundant Factory Caller Check
    - recoverIdentity Can Overwrite the newWallet's Existing Identity
    - Max Decimal Difference Check Can Be Moved to the Initialize
    - SaleFixPrice should have pause functionality

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit covers UFarm, a platform that turns real-world assets (RWAs) into tokens on EVM chains. It is built on the ERC-3643 permissioned token standard. Each asset is issued as an ERC-20 compatible security token, and only verified investors can hold or transfer it. Investors are verified off-chain (KYC/KYB/AML), and the rules are enforced on-chain through identity registries and a compliance module.

The review was conducted over a week period and included a comprehensive analysis of the relevant smart contracts. It covered the ERC-3643 token suite and its identity and compliance modules, the factory that deploys a full token suite for each asset, the private credit token with its investment and withdrawal settlement flows, the private credit value aggregator, and the fixed-price sale contract.

During the assessment, we did not identify any critical or high-severity issues. We identified one medium-severity finding: the withdrawal settlement flow does not take a holder's frozen token balance into account. As a result, settlements revert and holders with partially frozen balances cannot redeem or move their tokens until an agent removes the freeze. In addition, we identified one low-severity finding and five informational issues.

All identified issues were either remediated or acknowledged by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

▪ Review Led by

Trung Dinh, Lead Security Researcher

▪ Scope

The analyzed resources are located on:

https://github.com/UFarmDigital/UFarm-RWA-Contracts-private/tree/190cf8f19b45799e78dd29085ca9cc3e29c5877a

The issues described in this report were fixed in the following commit:

https://github.com/UFarmDigital/UFarm-RWA-Contracts/commit/5174ba8009af35cfdea19de0843e886ddfb3ff3c

▪ Changelog

16 September 2026	Audit start
25 September 2026	Initial report
30 September 2026	Revision received
12 October 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

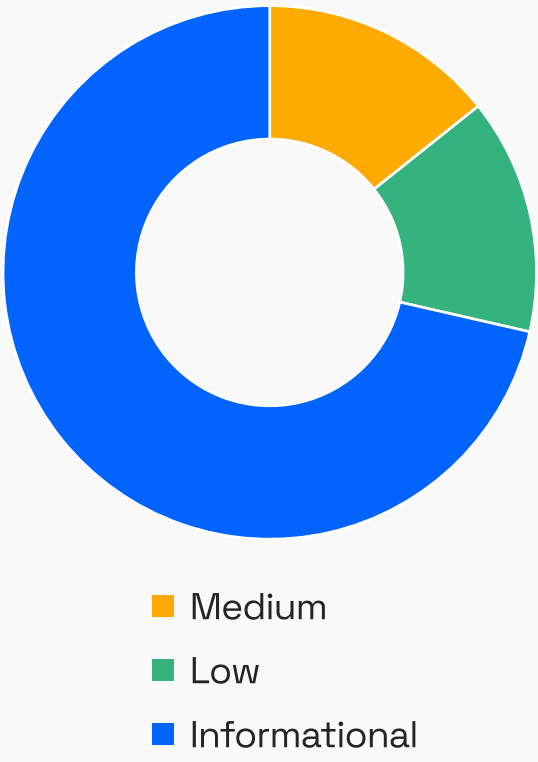
- **Issue Symbolic Codes**

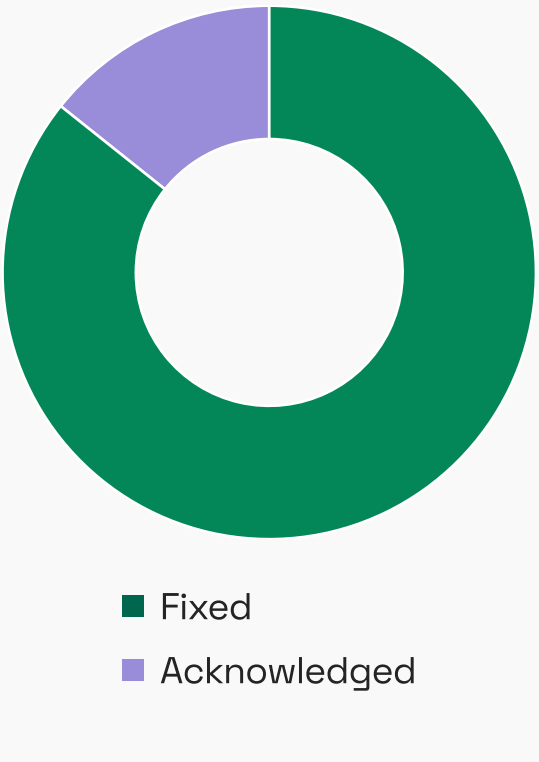
Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	1
Low	1
Informational	5
Total:	**7**





6. Weaknesses

This section contains the list of discovered weaknesses.

UFARM3-1 | Lack of frozen balance consideration in the withdrawal settlement flow

Fixed

Severity:	Probability:	Impact:
Medium	Rare	High

Path:

contracts/impl/PrivateCreditToken.sol#L544-L545
contracts/impl/PrivateCreditToken.sol#L725

Description:

The **PrivateCreditToken** settles a withdrawal by burning the holder's tokens through **settleWithdrawals()**:

```
state.reservedWithdrawal = 0;
state.withdrawalRequestedAt = 0;
_settling = true;
_burn(holder, amount);
```

The burn reaches the **to == address(0)** branch of **ERC3643Token._update()**, which validates the amount against the holder's unfrozen balance:

```
if (!_forceTransfer) {
    _requireNotFrozen(from);
    _requireFreeBalance(from, amount);
}
```

In other words, a settlement can only ever burn **balanceOf(holder) - _frozenTokens[holder]**. The withdrawal logic, however, never takes **_frozenTokens** into account when it decides how many tokens the settlement will burn, nor when it decides how many tokens a holder is allowed to transfer away. The two instances below describe how each of those omissions makes a settlement revert with **PartialFreezeExceeded**.

In both cases the outcome is the same: the holder's position can no longer be redeemed.

Instance 1: the matured settlement redeems balanceOf, not balanceOf - frozen

At or after maturity, **settleWithdrawals()** settles every supplied holder's full balance without requiring a withdrawal request. The amount is taken from **_settlementFundingRequired()**:

```
amount = isMatured
    ? balanceOf(holder)
    : state.reservedWithdrawal;
```

Since the matured branch passes **amount == balanceOf(holder)**, any nonzero **_frozenTokens[holder]** makes **_requireFreeBalance()** fail and the settlement reverts.

A partial freeze is meant to immobilise only the frozen portion of a position, but here a single frozen base unit makes the holder's entire matured position unredeemable. The holder has no way out on their own either: after maturity normal transfers revert with **MaturedTransfersDisabled**, so the unfrozen part can neither be redeemed nor moved, while income accrual has already stopped at maturity.

Instance 2: a transfer can push the reserved withdrawal into the frozen balance

_requestWithdrawal() deliberately keeps the reserved and the frozen portions of a position disjoint:

```
uint256 unreservedBalance = balanceOf(msg.sender) - state.reservedWithdrawal;
if (amount > unreservedBalance) revert ReservedBalanceExceeded();

_requireNotFrozen(msg.sender);
_requireFreeBalance(msg.sender, state.reservedWithdrawal + amount);
```

The last check requires **reservedWithdrawal + amount <= balanceOf - _frozenTokens**, which establishes the invariant **reservedWithdrawal + _frozenTokens <= balanceOf**. The settlement depends on that invariant holding, because the settlement burn is validated against the unfrozen balance.

_update() does not preserve it. The private-credit hook only checks the reserved portion:

```
if (
    !_recovering
        && amount > balanceOf(from) - fromState.reservedWithdrawal
) revert ReservedBalanceExceeded();
```

while **ERC3643Token._update()** only checks the frozen portion through **_requireFreeBalance(from, amount)**, that is **amount <= balanceOf - _frozenTokens**.

The two limits are evaluated independently over overlapping parts of the same balance, so an outgoing transfer is capped at **balanceOf - max(reserved, frozen)** instead of b**alanceOf - reserved - frozen**. The holder can therefore move out **min(reserved, frozen)** more than the invariant allows:

- the holder has a balance **x = 1000** with **y = 100** frozen, and reserves **z = 400**. The request is accepted because **400 + 100 <= 1000**.
- the holder transfers **x - z = 600** out. It passes the reserved check (**600 <= 1000 - 400**) and the free balance check (**600 <= 1000 - 100**).
- the holder is left with a balance of **400**, **400** reserved and **100** frozen, so **reserved + frozen = 500 > balance = 400**.

settleWithdrawals() then tries to burn the 400 reserved units while **_requireFreeBalance()** only sees **400 - 100 = 300** available, and reverts.

The reservation can never be settled. There is no entry point to cancel a withdrawal request, and because **reservedWithdrawal == balanceOf**, the holder can no longer transfer anything either, so the position becomes completely immobile.

Note that this is distinct from the case where an agent freezes tokens after a request has been made. Here no further agent action is required: the holder reaches the broken state themselves through an ordinary transfer, following a partial freeze that **_requestWithdrawal()** had explicitly validated as non-overlapping.

Remediation:

For the first instance, cap the matured settlement at the holder's free balance so that only the frozen units stay locked:

```
    amount = isMatured
--          ? balanceOf(holder)
++           ? balanceOf(holder) - _frozenTokens[holder]
            : state.reservedWithdrawal;
```

Note that **isFullExit** in **settleWithdrawals()** will then correctly evaluate to **false** while frozen units remain, so the partial exit branch handles the income crystallisation.

For the second instance, enforce the combined limit in **PrivateCreditToken._update()** so that an outgoing transfer can never eat into either lock:

```
    if (
        !_recovering
--          && amount > balanceOf(from) - fromState.reservedWithdrawal
++           && amount > balanceOf(from) - fromState.reservedWithdrawal - _frozenTokens[from]
    ) revert ReservedBalanceExceeded();
```

Furthermore consider also rejecting a freeze that would overlap an outstanding reservation in **freezePartialTokens()**, for example by requiring **nextFrozen + _holderState[wallet].reservedWithdrawal <= balance**, so that the same invariant cannot be broken from the agent side either.

UFARM3-7 | No cap on the total raise

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

contracts/impl/PrivateCreditToken.sol#L212-L223

Description:

mintInvestment() enforces a minimum ticket size, but there is no maximum raise.

Investors can keep minting as long as they pass the existing checks. The funds are sent directly to the funding address, so the borrower cannot reject an investment:

```
IERC20(_stablecoin).safeTransferFrom(msg.sender, _fundingAddress, stablecoinIn);
```

For example, a borrower may expect a $5M raise, but investors could potentially increase the debt to $50M. The additional $45M still accrues interest even if the borrower has no use for it.

This also increases the credit exposure of existing investors because the borrower's total repayment obligation can grow beyond the intended deal size.

Remediation:

Add a maximum raise to **DebtConditions** and enforce it during minting:

```
if (totalSupply() + amount > _debtConditions.maxSupply)
    revert RaiseCapExceeded();
```

The cap should be part of the disclosed debt terms so the maximum debt is known upfront.

Commentary from the client:

This scenario isn't likely in practice. While individual investments are not explicitly approved, participation is still permissioned, so only authorized investors can mint.

UFARM3-2 | forcedTransfer desyncs frozenTokens

Fixed

Severity:	Probability:	Impact:
Informational	Unlikely	Informational

Path:

contracts/impl/ERC3643Token.sol#L193-L202

Description:

forcedTransfer can move tokens that are partially frozen, but it does not decrease the sender's **_frozenTokens** counter.

As a result, **_frozenTokens[from]** can become larger than the wallet's actual balance. This breaks an accounting invariant used throughout the token and can prevent the wallet from transferring or redeeming its remaining tokens.

For example:

1. A wallet has **1000** tokens, all of which are frozen.
2. An agent calls **forcedTransfer(wallet, custodian, 1000)**.
3. The wallet now has **0** balance but still has **1000** frozen tokens recorded.
4. The wallet later receives **500** tokens.
5. Any outbound transfer or settlement reverts with **PartialFreezeExceeded**, because **frozenTokens (1000) > balance (500)**.

Remediation:

After **forcedTransfer**, clamp the sender's frozen amount to its remaining balance:

```
uint256 frozenAmount = _frozenTokens[from];
uint256 remaining = balanceOf(from);

if (frozenAmount > remaining) {
    _frozenTokens[from] = remaining;
    emit TokensFrozen(from, remaining);
}
```

This keeps **_frozenTokens[from] <= balanceOf(from)** after forced transfers.

UFARM3-3 | Redundant Factory Caller Check

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

contracts/impl/IdentityRegistryStorage.sol#L62
contracts/impl/Compliance.sol#L72

Description:

The functions **IdentityRegistryStorage.unbindRegistry()** and **Compliance.unbindToken()** use the **onlyOwnerOrFactory()** modifier, which allows the caller to be either the owner or the factory.

```
modifier onlyOwnerOrFactory()
    if (msg.sender != owner() && msg.sender != _factory) revert NotOwnerOrFactory();
    _;
}
```

However, the **TokenFactory** contract does not contain any functionality that invokes either of these functions.

As a result, the factory check in these two functions is redundant.

Remediation:

Consider changing the modifier from **onlyOwnerOrFactory** to **onlyOwner**:

```
-- function unbindToken(address token) external override onlyOwnerOrFactory {
++ function unbindToken(address token) external override onlyOwner {

-- function unbindRegistry(address registry) external onlyOwnerOrFactory {
++ function unbindRegistry(address registry) external onlyOwner {
```

UFARM3-4 | recoverIdentity Can Overwrite the newWallet's Existing Identity

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

contracts/impl/IdentityRegistry.sol#L121-L132

Description:

The **IdentityRegistry.recoverIdentity()** function is used to recover an identity from a lost wallet to a new wallet.

```
function recoverIdentity(address lostWallet, address newWallet) external override {
    // Registry storage is a trusted suite dependency; event ordering does not authorize state changes.
    // slither-disable-start reentrancy-events
    if (!_isAgent(msg.sender) && msg.sender != _boundToken) revert NotAgent();
    if (newWallet == address(0)) revert ZeroAddress();
    if (!_storage.contains(lostWallet)) revert NotFound();
    IIdentityRegistryStorage.InvestorData memory data = _storage.getInvestor(lostWallet);
    _storage.removeInvestor(lostWallet);
    _storage.storeInvestor(newWallet, data.identity, data.country);
    emit IdentityRecovered(lostWallet, newWallet, address(data.identity));
    // slither-disable-end reentrancy-events
}
```

The function directly stores the **lostWallet**'s identity data for **newWallet** without checking whether **newWallet** already has an existing identity.

As a result, an existing identity associated with **newWallet** can be overwritten without any notification or event indicating that its data has been replaced.

Remediation:

Consider either requiring **newWallet** to have no existing identity, or requiring its existing identity data to match the **lostWallet**'s data:

```
function recoverIdentity(address lostWallet, address newWallet) external override {
    // Registry storage is a trusted suite dependency; event ordering does not authorize state changes.
    // slither-disable-start reentrancy-events
    if (!_isAgent(msg.sender) && msg.sender != _boundToken) revert NotAgent();
```

```
    if (newWallet == address(0)) revert ZeroAddress();
++  if (_storage.contains(newWallet)) revert NotEmptyNewWallet();
    if (!_storage.contains(lostWallet)) revert NotFound();
    IIdentityRegistryStorage.InvestorData memory data = _storage.getInvestor(lostWallet);
    _storage.removeInvestor(lostWallet);
    _storage.storeInvestor(newWallet, data.identity, data.country);
    emit IdentityRecovered(lostWallet, newWallet, address(data.identity));
    // slither-disable-end reentrancy-events
}
```

Alternatively, if overwriting an existing identity is intentional, consider emitting an event that records the change to **newWallet**'s existing identity:

```
function recoverIdentity(address lostWallet, address newWallet) external override {
    // Registry storage is a trusted suite dependency; event ordering does not authorize state changes.
    // slither-disable-start reentrancy-events
    if (!_isAgent(msg.sender) && msg.sender != _boundToken) revert NotAgent();
    if (newWallet == address(0)) revert ZeroAddress();
    if (!_storage.contains(lostWallet)) revert NotFound();
    IIdentityRegistryStorage.InvestorData memory data = _storage.getInvestor(lostWallet);
++  IIdentityRegistryStorage.InvestorData memory newWalletOldData = _storage.getInvestor(newWallet);
    _storage.removeInvestor(lostWallet);
    _storage.storeInvestor(newWallet, data.identity, data.country);
    emit IdentityRecovered(lostWallet, newWallet, address(data.identity));
++  emit IdentityChanged(newWallet, newWalletOldData, data);
    // slither-disable-end reentrancy-events
}
```

UFARM3-5 | Max Decimal Difference Check Can Be Moved to the Initialize

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

contracts/impl/SaleFixPrice.sol#L174
contracts/impl/SaleFixPrice.sol#L179

Description:

In the **SaleFixPrice._convertDecimals()** function, there are checks to ensure that the difference between **fromDecimals** and **toDecimals** does not exceed **MAX_DECIMAL_DIFF**:

```
function _convertDecimals(
    uint256 amount,
    uint8 fromDecimals,
    uint8 toDecimals
) internal pure returns (uint256 converted) {
    if (toDecimals == fromDecimals) {
        return amount;
    }

    if (toDecimals > fromDecimals) {
        uint256 increase = uint256(toDecimals - fromDecimals);
        if (increase > MAX_DECIMAL_DIFF) revert DecimalDifferenceTooLarge(fromDecimals, toDecimals);
        return amount * (10 ** increase);
    }

    uint256 decrease = uint256(fromDecimals - toDecimals);
    if (decrease > MAX_DECIMAL_DIFF) revert DecimalDifferenceTooLarge(fromDecimals, toDecimals);

    uint256 factor = 10 ** decrease;
    if (amount % factor != 0) revert NonRepresentableAmount(amount, fromDecimals, toDecimals);
    return amount / factor;
}
```

This causes the decimal difference to be checked on every conversion. However, conversions are only performed between the sale token and the stablecoins configured in the **_stablecoins** storage array. Therefore, the decimal difference for each token pair can be validated once during initialization, eliminating the need to repeat this check on every conversion.

Remediation:

Consider moving the decimal difference check to the **initialize()** function before adding each stablecoin:

```
function initialize(address owner_, address token_, address[] calldata stablecoins_) external initializer {
    ...

    for (uint256 i = 0; i < stablecoins_.length; ++i) {
        address stablecoin = stablecoins_[i];
        if (stablecoin == address(0)) revert ZeroAddress();
        if (stablecoin == token_) revert InvalidStablecoin(stablecoin);
        if (_isStablecoin[stablecoin]) revert DuplicateStablecoin(stablecoin);

++      uint8 tokenDecimals = IERC20Metadata(token_).decimals();
++      uint8 stablecoinDecimals = IERC20Metadata(stablecoin).decimals();
++      if (stablecoinDecimals > tokenDecimals) {
++          uint256 increase = uint256(stablecoinDecimals - tokenDecimals);
++          if (increase > MAX_DECIMAL_DIFF) {
++              revert DecimalDifferenceTooLarge(tokenDecimals, stablecoinDecimals);
++          }
++      } else {
++          uint256 decrease = uint256(tokenDecimals - stablecoinDecimals);
++          if (decrease > MAX_DECIMAL_DIFF) {
++              revert DecimalDifferenceTooLarge(tokenDecimals, stablecoinDecimals);
++          }
++      }

        _isStablecoin[stablecoin] = true;
        _stablecoins.push(stablecoin);
    }
}
```

The corresponding checks can then be removed from **_convertDecimals()**:

```
function _convertDecimals(
    uint256 amount,
    uint8 fromDecimals,
    uint8 toDecimals
) internal pure returns (uint256 converted) {
    if (toDecimals == fromDecimals) {
        return amount;
    }

    if (toDecimals > fromDecimals) {
```

```
        uint256 increase = uint256(toDecimals - fromDecimals);
--      if (increase > MAX_DECIMAL_DIFF) revert DecimalDifferenceTooLarge(fromDecimals, toDecimals);
        return amount * (10 ** increase);
    }

    uint256 decrease = uint256(fromDecimals - toDecimals);
--  if (decrease > MAX_DECIMAL_DIFF) revert DecimalDifferenceTooLarge(fromDecimals, toDecimals);

    uint256 factor = 10 ** decrease;
    if (amount % factor != 0) revert NonRepresentableAmount(amount, fromDecimals, toDecimals);
    return amount / factor;
}
```

This preserves the invariant while avoiding the repeated **MAX_DECIMAL_DIFF** checks during subsequent conversions.

UFARM3-6 | SaleFixPrice should have pause functionality

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

contracts/impl/SaleFixPrice.sol

Description:

The **SaleFixPrice** contract enables users to swap between the sale token and whitelisted stablecoins at a fixed 1:1 rate via the **buyWith()** and **sellFor()** functions. However, the contract lacks administrative controls to pause trading or delist specific stablecoins.

If a supported stablecoin experiences a severe depeg event (similar to UST), an attacker can sell the depegged stablecoin to get the contract's sale token reserves. They can then convert those sale tokens into other assets, resulting in significant protocol losses. Without a pause or delisting mechanism, the protocol has no emergency method to mitigate losses in real time.

Remediation:

Consider implementing owner-controlled functions to:

1. Pause and unpause **buyWith()** and **sellFor()** (e.g., using OpenZeppelin's **Pausable**).
2. Remove or disable specific stablecoins from the allowlist.

hexens

x

UFARM DIGITAL